
RAPPORT DE GÉNIE LOGICIEL ET CONDUITE DE PROJET

Application des concepts de Génie Logiciel
et de Conduite de Projet au développement de
SaaS Directory

Présenté par :

**Omar SARSAR
Iliass HARIZ**

2026

*Nous dédions ce travail à nos familles pour leur soutien indéfectible,
à nos enseignants pour leur guidance et leurs enseignements,
et à tous ceux qui croient en l'innovation technologique au service de la
communauté.*

Nous tenons à exprimer notre profonde gratitude envers l'ensemble de nos enseignants qui nous ont accompagnés tout au long de notre parcours académique. Leurs conseils avisés, leur patience et leur exigence ont été des facteurs déterminants dans la réussite de ce projet.

Nous remercions également nos camarades de promotion pour les échanges enrichissants et l'entraide collective qui ont contribué à notre progression.

Ce rapport présente l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de **SaaS Directory**, une plateforme de marketplace dédiée aux produits logiciels en mode SaaS. Le projet a été réalisé avec une stack technologique moderne comprenant Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL et Better Auth 1.6.9.

L'étude couvre l'ensemble des neuf chapitres du cours de Génie Logiciel : rappels fondamentaux, généralités, cycle de vie du logiciel, démarche de conception, normalisation, tests, gestion de projet, analyse des risques et conduite du changement. Chaque concept théorique est illustré par sa mise en oeuvre concrète dans le projet.

Le rapport inclut une estimation du coût par la méthode COCOMO appliquée à la base de code réelle de 12 166 lignes, une analyse des risques liés aux breaking changes de Next.js 16.2.4, ainsi qu'une présentation détaillée de la gestion des versions via Git et des migrations Drizzle ORM.

Mots-clés : Génie Logiciel, Conduite de Projet, Next.js, React, Drizzle ORM, PostgreSQL, CO-COMO, Cycle de Vie, UML, Normalisation, Tests Logiciels, Gestion des Risques.

This report presents the systematic application of Software Engineering and Project Management concepts to the development of **SaaS Directory**, a marketplace platform dedicated to SaaS software products. The project was built using a modern technology stack including Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL, and Better Auth 1.6.9.

The study covers all nine chapters of the Software Engineering course : fundamental recalls, general concepts, software life cycle, design methodology, standardization, testing, project management, risk analysis, and change management. Each theoretical concept is illustrated by its concrete implementation in the project.

The report includes a COCOMO cost estimation applied to the actual codebase of 12,166 lines, a risk analysis related to Next.js 16.2.4 breaking changes, and a detailed presentation of version control via Git and Drizzle ORM migrations.

Keywords : Software Engineering, Project Management, Next.js, React, Drizzle ORM, PostgreSQL, COCOMO, Life Cycle, UML, Standardization, Software Testing, Risk Management.

Remerciements	3
Résumé	4
Abstract	5
Introduction Générale	11
1 Chapitre 1 : Rappels	12
1.1 Définitions Fondamentales	12
1.1.1 Logiciel et Application	12
1.1.2 Système d'Information (SI)	12
1.1.3 Méthode, Outil et Méthodologie	12
1.2 Application à SaaS Directory	13
2 Chapitre 2 : Génie Logiciel — Généralités	13
2.1 Objectifs du Génie Logiciel	13
2.2 Les Principes du GL Appliqués	13
2.2.1 Rigueur	13
2.2.2 Séparation des Problèmes	13
2.2.3 Modularité	13
2.2.4 Abstraction	14
2.2.5 Généricity	14
2.2.6 Construction Incrémentale	14
2.2.7 Anticipation du Changement	14
2.3 Les Attributs du Logiciel	14
2.3.1 Couplage	14
2.3.2 Complexité	14
2.3.3 Extensibilité	15
2.3.4 Lisibilité et Visibilité du Comportement	15

3	Chapitre 3 : Représentations du Cycle de Vie du Logiciel	15
3.1	Les Modèles de Cycle de Vie	15
3.2	Choix et Justification du Modèle	15
3.3	Application au Projet SaaS Directory	15
4	Chapitre 4 : La Démarche	16
4.1	Méthodes de Développement Semi-Formelles	16
4.2	Diagrammes UML Appliqués au Projet	16
4.2.1	Diagramme de Cas d'Utilisation	17
4.2.2	Diagramme de Classes	17
4.2.3	Diagramme de Séquence	17
4.3	Méthodes Formelles et Langage Z	17
5	Chapitre 5 : La Normalisation	17
5.1	La Norme ISO et la Certification ISO 9000	20
5.2	Les Normes IEEE	20
5.3	Normalisation Appliquée au Projet	20
5.3.1	Normes de Codage	20
5.3.2	Architecture Normalisée	20
5.3.3	Gestion Documentaire et Traçabilité	21
6	Chapitre 6 : Les Tests	21
6.1	Types de Tests et Stratégie	21
6.1.1	Tests Unitaires	21
6.1.2	Tests d'Intégration et End-to-End	21
6.1.3	Tests de Performance	21
6.2	Outils et Environnement de Test	21
6.3	Couverture et Résultats	22
7	Chapitre 7 : La Gestion des Projets Logiciels	22
7.1	Organisation du Projet (WBS, PBS, OBS)	22

7.2	Planification (GANTT, PERT)	22
7.3	Estimation des Coûts par COCOMO	22
7.3.1	Taille du Projet	23
7.3.2	Application du COCOMO de Base	23
8	Chapitre 8 : L'Analyse des Risques	24
8.1	Identification des Risques	24
8.1.1	Risques Techniques	24
8.1.2	Risques de Sécurité	24
8.1.3	Risques de Performance	24
8.2	Analyse et Stratégies de Mitigation	24
8.2.1	Mitigation des Risques Techniques	24
8.2.2	Mitigation des Risques de Sécurité	25
8.2.3	Mitigation des Risques de Performance	25
8.3	Plan de Continuité	25
9	Chapitre 9 : Conduite du Changement	25
9.1	Nature des Changements	25
9.2	Gestion des Versions	25
9.3	Adaptation et Communication	26
	Conclusion Générale et Perspectives	26
	Bibliographie et Webographie	27
	Annexes	28
	Annexe A : Schema de la Base de Donnees	28
	Annexe C : Dependances Principales	31
	Annexe D : Diagramme de Deploiement	32

1	Diagramme de Cas d’Utilisation — SaaS Directory	18
2	Diagramme de Classes (20 tables)	19
3	Diagramme de Séquence — Envoi d’un Message	19
4	Diagramme de GANTT — Planification du Projet SaaS Directory	23
5	Diagramme de Deploiement — Architecture Physique de SaaS Directory	32

1	Estimation COCOMO de Base — Mode Semi-Détaché (14 KLOC)	23
2	Dependances principales du projet SaaS Directory	31

Dans un monde de plus en plus numérisé, le développement de logiciels est devenu une discipline complexe qui exige rigueur, méthodologie et gestion efficace des ressources. Le Génie Logiciel (GL) et la Conduite de Projet fournissent le cadre théorique et pratique nécessaire pour mener à bien des projets logiciels de qualité, dans les délais et budgets impartis.

Ce rapport a pour objectif de démontrer l'application concrète des concepts théoriques du Génie Logiciel et de la Conduite de Projet au développement de **SaaS Directory**, une application web moderne permettant aux créateurs de logiciels de lancer, promouvoir et vendre leurs produits SaaS.

SaaS Directory est une plateforme de marketplace dédiée aux produits logiciels en mode SaaS (Software as a Service). L'application offre un catalogue où les utilisateurs peuvent découvrir des outils logiciels, voter pour leurs produits préférés, interagir via des commentaires et une messagerie, et souscrire à des plans d'abonnement Free ou Pro.

Le projet a été réalisé avec une stack technologique contemporaine et cohérente : Next.js 16.2.4 (framework React avec App Router), React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2 (ORM type-safe pour PostgreSQL), Better Auth 1.6.9 (authentification complète avec OAuth et 2FA), Redis (cache côté serveur), et Playwright 1.59.1 pour les tests end-to-end.

Le codebase compte exactement **20 tables** dans le schéma Drizzle ORM, **12 166 lignes de code** réparties sur **108 fichiers** TypeScript/TSX, avec **4 migrations** versionnées (0000 à 0003). Le projet est versionné sur Git avec une branche master principale et des branches de fonctionnalités (omar, redis).

Ce rapport s'articule autour de neuf chapitres principaux. Le premier chapitre présente les rappels fondamentaux sur les concepts de base. Le deuxième traite des généralités du Génie Logiciel et de ses principes. Le troisième décrit les modèles de cycle de vie. Le quatrième expose la démarche de conception avec UML. Le cinquième aborde la normalisation. Le sixième détaille les stratégies de tests. Le septième présente la gestion de projet et l'estimation COCOMO. Le huitième analyse les risques. Le neuvième et dernier chapitre traite de la conduite du changement.

1.1 Définitions Fondamentales

Avant d'aborder les concepts avancés du Génie Logiciel, il est essentiel de poser les bases terminologiques et conceptuelles.

1.1.1 Logiciel et Application

Un **logiciel** est défini comme un ensemble de programmes, procédures, règles et documentations relatifs au fonctionnement d'un ensemble de traitements d'informations. Une **application** est un logiciel qui sert à implanter un système d'informations. On distingue trois grandes catégories : applications de gestion (peu de traitement, beaucoup de données), applications scientifiques (beaucoup de traitement, peu de données), et applications industrielles (peu de traitement et de données).

SaaS Directory s'inscrit dans la catégorie des applications de gestion/web, avec une composante significative de données structurées (20 tables relationnelles).

1.1.2 Système d'Information (SI)

Le **Système d'Information (SI)** constitue l'ensemble des flux d'opérations, d'informations et de moyens mis en oeuvre par une organisation. Le **Système d'Information Automatisé (SIA)** repose sur la technologie informatique. SaaS Directory est un SIA complet : il traite des profils utilisateurs, des produits, des votes, des messages et des abonnements via une infrastructure web moderne.

1.1.3 Méthode, Outil et Méthodologie

Une méthode est un processus discipliné qui génère un ensemble de modèles décrivant les différents aspects d'un système logiciel, en utilisant une notation bien définie.

Un **outil** désigne un programme ou un ensemble de programmes aidant à la mise en oeuvre des techniques. L'**Atelier de Génie Logiciel (AGL)** regroupe l'ensemble des outils utilisés pour le développement, la maintenance et la documentation.

La **méthodologie** est la démarche générale qui fait appel à des méthodes et des outils pour aboutir au logiciel final. Pour SaaS Directory, la méthodologie adoptée combine l'approche incrémentale itérative avec les principes du cycle de vie en V.

1.2 Application à SaaS Directory

SaaS Directory s'inscrit dans le cadre théorique des Systèmes d'Information Automatisés. En tant que marketplace web, il traite de grands volumes de données structurées et exige une architecture logicielle rigoureuse pour garantir performance, sécurité et évolutivité.

2.1 Objectifs du Génie Logiciel

Le **Génie Logiciel (GL)** vise à utiliser l'informatique dans les entreprises comme outil quotidien, à réduire les coûts de développement et de maintenance, à améliorer la qualité des logiciels, et à augmenter la productivité des équipes. Ces objectifs s'inscrivent dans une démarche globale de maîtrise du cycle de vie du logiciel.

2.2 Les Principes du GL Appliqués

2.2.1 Rigueur

La rigueur se traduit par l'utilisation de TypeScript (typage statique strict), les conventions de nommage systématiques (PascalCase pour les composants, camelCase pour les fonctions), et la revue de code via les pull requests Git. Le fichier `AGENTS.md` (327 octets) documente les conventions spécifiques du projet.

2.2.2 Séparation des Problèmes

Trois niveaux de séparation sont appliqués : séparation dans le temps (développement par phases avec tests à chaque niveau), séparation dans l'espace (architecture en couches : Client Components, Server Actions, Drizzle ORM, PostgreSQL), et séparation en sous-systèmes (modules `src/app/`, `src/components/`, `src/lib/`).

2.2.3 Modularité

L'architecture repose sur des composants React indépendants, chacun responsable d'une fonctionnalité spécifique. Le schéma Drizzle ORM décompose le système en 20 tables distinctes, chacune encapsulant une entité métier. Cette modularité facilite la maintenance et les tests unitaires.

2.2.4 Abstraction

Plusieurs niveaux d'abstraction sont utilisés : les Server Actions abstraient la logique métier du rendu UI ; Drizzle ORM abstrait les requêtes SQL complexes ; le système de cache Redis abstrait la gestion des données temporaires.

2.2.5 Généricity

Les composants UI sont conçus comme des composants réutilisables paramétrables via les props React. Le système d'authentification Better Auth est générique et supporte OAuth (Google, GitHub) et l'authentification par email/mot de passe.

2.2.6 Construction Incrémentale

Le développement a suivi une approche incrémentale documentée dans l'historique Git. Les phases principales sont : fondations (Next.js 16.2.4, shadcn, Drizzle), infrastructure de tests (Vitest, Playwright), base de données (schéma Drizzle, migrations, seed), authentification TDD (login/signup, tests), fonctionnalités core (marketplace, profils, messages, notifications), et polissage final (recherche, responsive, build fixes).

2.2.7 Anticipation du Changement

L'architecture anticipe plusieurs évolutions : le schéma Drizzle inclut déjà les champs `stripeCustomerId` et `stripeSubscriptionId` pour une future intégration Stripe ; le système de plans (Free/Pro) permet d'ajouter de nouveaux niveaux d'abonnement.

2.3 Les Attributs du Logiciel

L'analyse du codebase de 12 166 lignes réparties sur 108 fichiers permet d'évaluer ces attributs.

2.3.1 Couplage

Le couplage est faible grâce à l'architecture par composants React et l'encapsulation des Server Actions. Chaque table Drizzle ORM est indépendante, avec des relations explicites via les clés étrangères. Le couplage entre frontend (React) et backend (Server Actions) est réduit au minimum par l'API interne de Next.js 16.2.4.

2.3.2 Complexité

Avec 20 tables relationnelles et 12 166 lignes de code, le système présente une complexité modérée. Cette complexité est maîtrisée par la modularité, le typage statique TypeScript, et la documentation

inline (JSDoc).

2.3.3 Extensibilité

L’extensibilité est démontrée par la facilité d’ajouter de nouvelles tables au schéma Drizzle (4 migrations versionnées), l’ajout de nouveaux composants UI sans impacter les existants, et la configuration modulaire de Next.js (`next.config.ts`).

2.3.4 Lisibilité et Visibilité du Comportement

La lisibilité est assurée par TypeScript (typage explicite), les conventions de nommage strictes, et la documentation JSDoc. La visibilité du comportement est garantie par les tests unitaires (10 fichiers de test couvrant l’authentification, la souscription, le schéma, les actions serveur) et les tests end-to-end (Playwright 1.59.1).

Le développement d’un logiciel n’est pas simplement l’écriture d’un programme. La taille et la complexité des logiciels actuels exigent l’utilisation de processus de développement structurés, appelés **cycles de vie du logiciel**.

3.1 Les Modèles de Cycle de Vie

Le modèle en cascade (waterfall) suit une séquence linéaire de phases : spécification, conception, codage, tests, et maintenance. Le modèle en V associe à chaque phase de développement une phase de test correspondante. Le modèle itératif et incrémental divise le projet en itérations courtes, chacune livrant une version fonctionnelle du logiciel.

3.2 Choix et Justification du Modèle

Pour SaaS Directory, nous avons adopté un **modèle hybride** combinant l’approche incrémentale itérative avec des éléments du modèle en V. Cette décision repose sur trois arguments : la nature évolutive du projet (marketplace web), les contraintes temporelles académiques, et l’importance de la validation systématique (tests à chaque niveau).

3.3 Application au Projet SaaS Directory

Le développement de SaaS Directory s’est déroulé en plusieurs phases documentées dans l’historique Git, chacune apportant des fonctionnalités concrètes.

Phase 1 : Fondations. Initialisation du projet Next.js 16.2.4 avec TypeScript et Tailwind CSS v4, configuration de Drizzle ORM 0.45.2 avec PostgreSQL, mise en place de Better Auth 1.6.9, et création des tables de base (user, session, account, verification).

Phase 2 : Infrastructure de tests. Mise en place de Vitest 4.1.5, React Testing Library, Playwright 1.59.1, et configuration des seuils de couverture.

Phase 3 : Base de données et schéma. Définition du schéma Drizzle ORM complet (20 tables), création des 4 migrations versionnées, et script de seed.

Phase 4 : Authentification et profils. Développement des pages login/signup avec TDD, gestion des profils utilisateurs (table profile), et système de lancement de produits (tables launch, category, launchCategory).

Phase 5 : Fonctionnalités core. Développement du marketplace (votes, commentaires), messagerie (tables conversation, message, notification), et système d'abonnements Free/Pro (tables plan, subscription).

Phase 6 : Polissage et optimisation. Ajout de la recherche, reset de mot de passe, corrections responsive, et build fixes.

La démarche de conception d'un logiciel définit la méthodologie suivie pour passer du besoin exprimé au logiciel opérationnel.

4.1 Méthodes de Développement Semi-Formelles

Le cours présente plusieurs méthodes semi-formelles : SADT (Structure Analysis and Design Technique), MERISE (méthode française systémique avec séparation données/traitements), UML/Processus Unifié (itératif, incrémental, orienté objet), OMT (Object Modelling Technique de James Rumbaugh), et OOSE (Object-Oriented Software Engineering d'Ivar Jacobson).

Pour SaaS Directory, nous avons retenu l'approche UML/Processus Unifié pour plusieurs raisons : son caractère itératif et incrémental correspond à notre modèle de cycle de vie ; son orientation objet s'aligne avec notre stack technologique (React composants, Drizzle ORM entités) ; et UML est le standard industriel de facto pour la modélisation orientée objet.

4.2 Diagrammes UML Appliqués au Projet

4.2.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation modélise les interactions entre les acteurs et le système. Les acteurs principaux sont : Visiteur, Utilisateur Authentifié, Créateur de Produit, et Administrateur.

4.2.2 Diagramme de Classes

Le schéma Drizzle ORM dans `src/lib/db/schema.ts` constitue l'implémentation directe du diagramme de classes. Le système compte 20 tables.

Les tables sont regroupées en trois catégories : **Better Auth** (user, session, account, verification); **métier principal** (profile, launch, category, plan, subscription); et **interaction sociale** (upvote, comment, post, postUpvote, postComment, conversation, conversationParticipant, message, notification).

4.2.3 Diagramme de Séquence

Le diagramme de séquence illustre l'interaction entre les acteurs et le système lors de l'envoi d'un message.

L'utilisateur émetteur saisit le message via l'interface React. Le Client Component appelle la Server Action `sendMessage` qui insère le message dans la table `message` via Drizzle ORM. Cette architecture démontre la séparation des préoccupations entre présentation, métier et persistance.

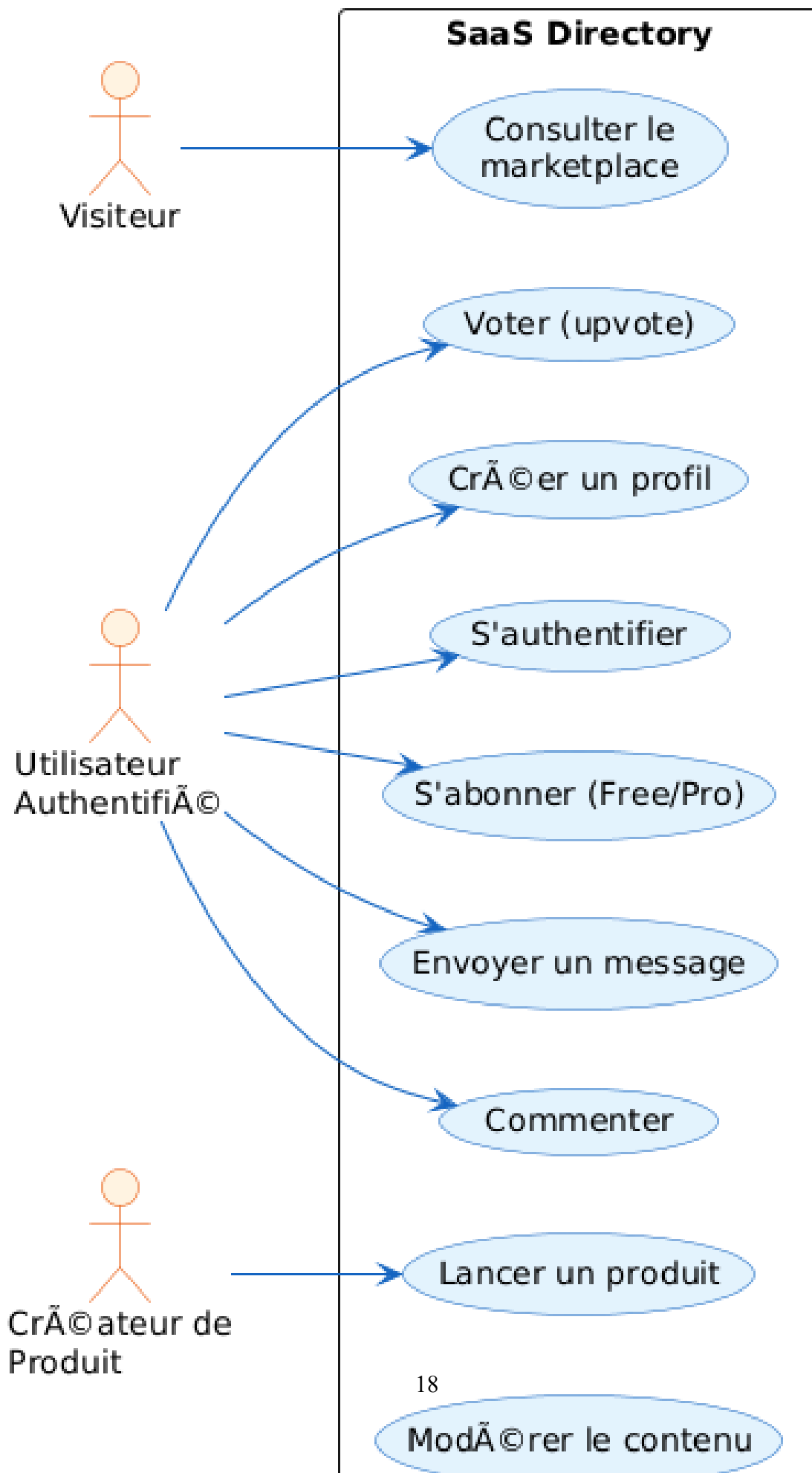
4.3 Méthodes Formelles et Langage Z

Les méthodes formelles (langage Z, B, VDM) utilisent des notations mathématiques pour spécifier, développer et vérifier des systèmes logiciels. Le langage Z, développé par Jean-Raymond Abrial, est basé sur la théorie des ensembles et la logique des prédicats.

Ces méthodes n'ont pas été utilisées dans ce projet pour plusieurs raisons pragmatiques : le domaine du marketplace web n'est pas critique au sens des systèmes embarqués ; la courbe d'apprentissage est élevée ; et TypeScript avec Zod fournissent un niveau de vérification suffisant (typage statique, validation de schémas).

Les normes sont des apports documentés contenant des spécifications techniques destinées à être utilisées systématiquement comme règles ou lignes directrices.

Please use 'option handwritten true' to enable handwritten



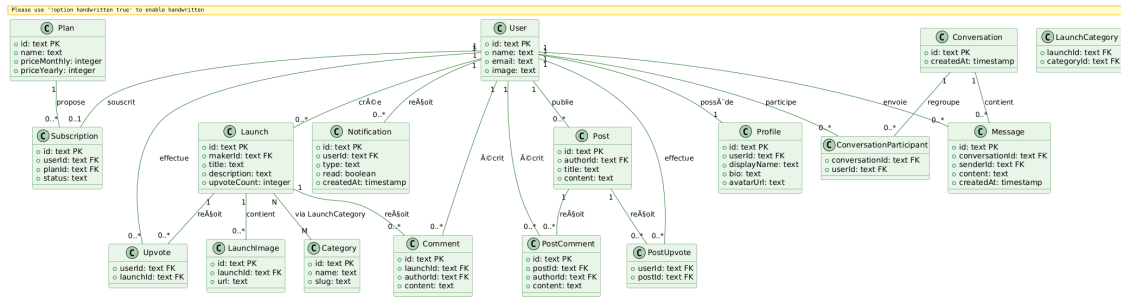


FIGURE 2 – Diagramme de Classes (20 tables)

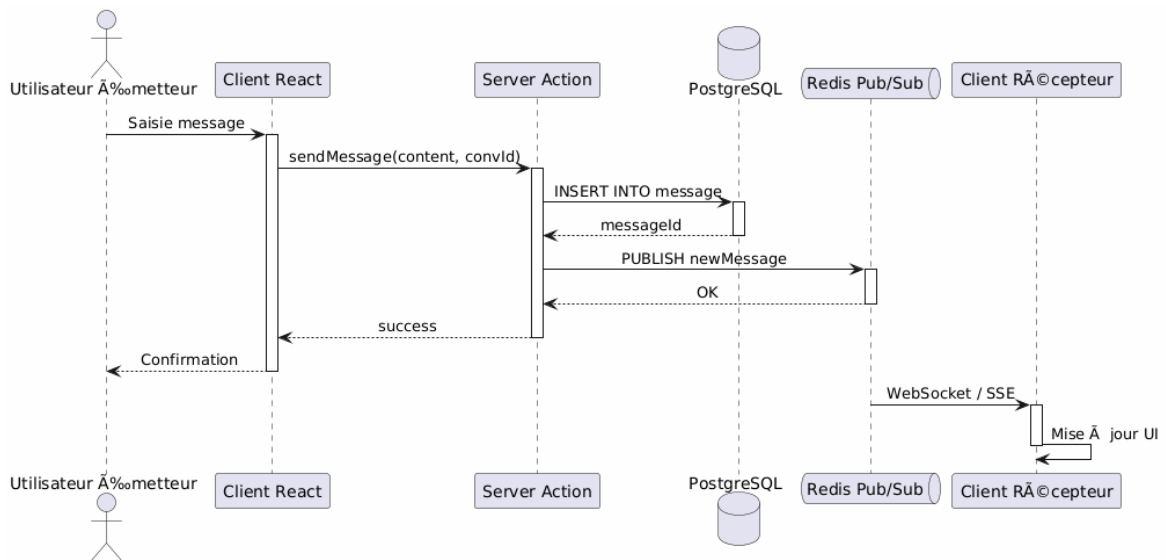


FIGURE 3 – Diagramme de Séquence — Envoi d'un Message

5.1 La Norme ISO et la Certification ISO 9000

L'International Organization for Standardization (ISO), créée en 1947, est une fédération mondiale d'organismes nationaux de normalisation. La norme ISO 9000 traduit la démarche qualité au niveau de l'entreprise. Dans le contexte de SaaS Directory, la gestion documentaire est assurée par les fichiers README.md et AGENTS.md, et la traçabilité est garantie par l'historique Git et les migrations Drizzle numérotées (0000 à 0003).

5.2 Les Normes IEEE

L'Institute of Electrical and Electronics Engineers (IEEE) a défini un ensemble de normes s'intéressant aux processus, produits et techniques de base du génie logiciel.

Fixe la signification des termes et choisit les processus prioritaires du cycle de vie du logiciel. Peut être considérée comme un dictionnaire et une liste de vérification pour la gestion des projets.

Recommandée pour les spécifications des exigences du logiciel. Pourrait être intégrée avec les approches de validation par prototype.

Pour appliquer la validation et la vérification. Norme facilement adaptable à tous les problèmes.

5.3 Normalisation Appliquée au Projet

5.3.1 Normes de Codage

Le projet respecte des conventions strictes : PascalCase pour les composants React (UserCard.tsx), camelCase pour les fonctions (sendMessage()), et kebab-case pour les routes API. Les imports sont organisés par groupe : React/Next.js, bibliothèques tierces, composants internes, utilitaires.

5.3.2 Architecture Normalisée

L'architecture suit le pattern MVC adapté à Next.js 16.2.4. Le **Modèle** est constitué par le schéma Drizzle ORM. La **Vue** est implémentée par les composants React. Le **Contrôleur** est assuré par les Server Actions dans src/lib/actions/.

5.3.3 Gestion Documentaire et Traçabilité

La gestion documentaire est assurée par plusieurs artefacts : README.md (présentation et installation), AGENTS.md (conventions pour les agents de développement), package.json (dépendances avec versions exactes), et commits Git suivant une convention descriptive (feat:, fix:, chore:).

Les tests logiciels constituent une phase critique du cycle de vie du logiciel, visant à détecter les erreurs et à garantir que le système répond aux exigences spécifiées.

6.1 Types de Tests et Stratégie

La stratégie de tests de SaaS Directory repose sur une pyramide de tests classique.

6.1.1 Tests Unitaires

Les tests unitaires sont implémentés avec Vitest 4.1.5. Dix fichiers de test couvrent l'authentification (auth-client.test.ts), la souscription (subscription.test.ts), le schéma Drizzle (schema.test.ts), les actions serveur (post.test.ts, launch.test.ts, message.test.ts), et les composants UI (theme-toggle.test.tsx, infrastructure.test.tsx, page.test.tsx pour login et signup).

6.1.2 Tests d'Intégration et End-to-End

Les tests end-to-end sont réalisés avec Playwright 1.59.1. La configuration (playwright.config.ts) définit un projet Chromium avec tracing activé. Le fichier e2e/homepage.spec.ts teste le parcours utilisateur critique.

6.1.3 Tests de Performance

La performance est évaluée par l'optimisation des requêtes Drizzle ORM et la configuration des index PostgreSQL. Le projet inclut un script de benchmark pour le cache Redis (repertoire scripts/).

6.2 Outils et Environnement de Test

L'environnement de test de SaaS Directory utilise le fichier docker-compose.test.yml qui définit une base PostgreSQL de test sur le port 5433. La séparation des environnements (développement, test, production) est assurée par les variables d'environnement : DATABASE_URL pour le

développement, `DATABASE_URL_TEST` pour les tests.

6.3 Couverture et Résultats

La couverture de tests cible les fonctions critiques : authentification (création de compte, connexion, 2FA), gestion des rôles, marketplace (création de produit, vote, commentaire), messagerie (envoi, réception), et abonnements (souscription, changement de plan). Les seuils de couverture sont configurés dans `vitest.config.ts` avec un minimum de 70%.

La gestion des projets logiciels vise à structurer méthodiquement une réalité à venir, en appliquant un objectif et des actions à entreprendre avec des ressources données.

7.1 Organisation du Projet (WBS, PBS, OBS)

Trois outils structurants ont guidé l'organisation du projet.

Le **Work Breakdown Structure (WBS)** décompose le projet en tâches hiérarchiques : fondations, infrastructure de tests, base de données, authentification, fonctionnalités core, et polissage.

Le **Product Breakdown Structure (PBS)** identifie les livrables : authentification fonctionnelle, profil utilisateur opérationnel, marketplace avec votes et commentaires, système de messagerie, et abonnements Free/Pro.

Le **Organizational Breakdown Structure (OBS)** répartit les responsabilités entre les deux développeurs du projet.

7.2 Planification (GANTT, PERT)

La figure 4 présente la répartition temporelle des phases du projet sur une échelle de 130 jours. Le diagramme PERT modélise les dépendances entre tâches de type “fin à début”.

7.3 Estimation des Coûts par COCOMO

L'estimation des coûts d'un projet logiciel est un exercice délicat. Pour contrer les biais classiques (optimisme, négligence technique), nous avons appliqué la méthode COCOMO (CONstructive COSt MOdel) de Barry Boehm.

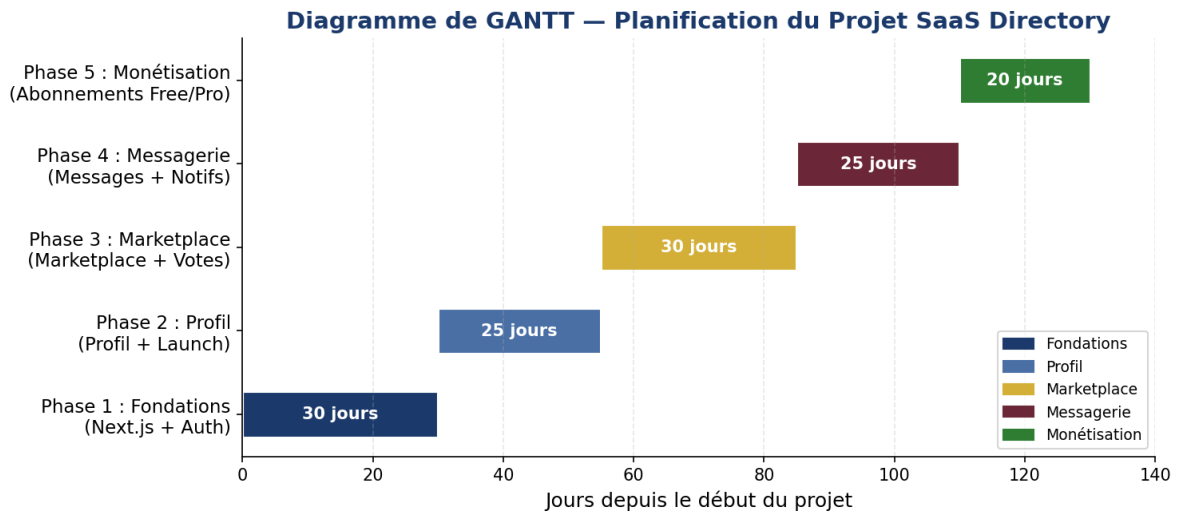


FIGURE 4 – Diagramme de GANTT — Planification du Projet SaaS Directory

7.3.1 Taille du Projet

Le codebase de SaaS Directory compte 12 166 lignes de code réparties sur 108 fichiers TypeScript/TSX, soit approximativement 14 KLOC. Répartition principale : `src/app` (3 198 LOC), `src/components` (5 234 LOC), `src/lib` (2 957 LOC), `tests` (111 LOC), `e2e` (33 LOC).

7.3.2 Application du COCOMO de Base

Pour un projet web moderne (Next.js 16.2.4, React 19.2.4, TypeScript 5), nous avons retenu le mode semi-détaché. Les coefficients sont $a = 3,0$ et $b = 1,12$.

TABLE 1 – Estimation COCOMO de Base — Mode Semi-Détaché (14 KLOC)

Paramètre	Formule / Valeur	Résultat
Taille du projet	12 166 LOC $\approx$ 14 KLOC	14 KLOC
Mode de développement	Semi-Détaché	$a = 3,0$; $b = 1,12$
Effort estimé	$E = a \times (\text{KLOC})^b$	$E \approx 58$ Personnes-Mois
Durée estimée	$D = 2,5 \times E^{0,35}$	$D \approx 9,5$ mois
Taille d'équipe théorique	$N = E/D$	$N \approx 6$ personnes
Taille d'équipe réelle	2 développeurs sur 5 mois	10 Personnes-Mois

Ces chiffres constituent une estimation théorique à visée pédagogique. En réalité, le projet a été réalisé par deux développeurs sur environ 10 Personnes-Mois, illustrant l'écart fréquent entre les modèles prédictifs et la réalité des projets académiques.

L'analyse des risques est une composante essentielle de la conduite de projet. Elle vise à identifier, évaluer et traiter les événements incertains qui pourraient affecter négativement les objectifs.

8.1 Identification des Risques

Trois catégories principales de risques ont été identifiées.

8.1.1 Risques Techniques

Le risque technique majeur réside dans la complexité de Next.js 16.2.4 avec le App Router. Ce framework introduit des breaking changes significatifs par rapport aux versions précédentes. Un risque secondaire concerne la configuration des relations et indexes Drizzle ORM 0.45.2 + PostgreSQL, qui nécessite une expertise SQL approfondie pour garantir les performances.

8.1.2 Risques de Sécurité

Les risques de sécurité concernent la protection des données utilisateurs et la prévention des attaques. L'injection SQL est mitigée par l'utilisation de Drizzle ORM avec des requêtes paramétrées. Les risques liés à l'authentification incluent la fuite de tokens JWT et la compromission des sessions. Better Auth 1.6.9 gère nativement plusieurs de ces menaces (expiration des tokens, rotation, rate limiting).

8.1.3 Risques de Performance

Le marketplace doit supporter un grand nombre de produits sans dégradation du temps de réponse, ce qui nécessite l'optimisation des indexes PostgreSQL et la pagination côté serveur. Le cache Redis est utilisé pour réduire la charge sur la base de données pour les données fréquemment accédées.

8.2 Analyse et Stratégies de Mitigation

8.2.1 Mitigation des Risques Techniques

La mitigation repose sur trois piliers : veille technologique (documentation officielle de Next.js consultée régulièrement), tests end-to-end Playwright 1.59.1 pour détecter les régressions, et modularité de l'architecture facilitant le remplacement d'une technologie si elle devient obsolète.

8.2.2 Mitigation des Risques de Sécurité

La sécurité est assurée par plusieurs mécanismes : Better Auth gère la sécurité des sessions (tokens JWT, expiration, rotation); les Server Actions valident les entrées utilisateur via Zod 4.4.3; et les indexes PostgreSQL accélèrent les requêtes fréquentes tout en limitant l'attaque par injection.

8.2.3 Mitigation des Risques de Performance

La performance est optimisée par : les indexes PostgreSQL définis dans le schéma Drizzle, le cache Redis pour les données fréquemment accédées (sessions, profils, listes de catégories), et la pagination côté serveur pour le marketplace et la messagerie.

8.3 Plan de Continuité

En cas d'incident majeur, plusieurs mesures sont prévues : restauration depuis les backups PostgreSQL, reconstruction du schéma via les migrations Drizzle (0000 à 0003), retour à une version stable via Git (branche `master` + tags), et documentation technique (README.md, AGENTS.md) facilitant l'onboarding d'un nouveau développeur.

La conduite du changement est la discipline qui vise à accompagner les transformations au sein d'un projet.

9.1 Nature des Changements

Plusieurs types de changements ont affecté le projet. Les **changements technologiques** incluent la mise à jour de Next.js 15 vers 16.2.4 (breaking changes sur le App Router), la migration de Tailwind CSS v3 vers v4, et l'ajout de Redis pour le cache côté serveur. Les **changements fonctionnels** concernent l'ajout de fonctionnalités : système de notifications, édition et suppression des commentaires, pagination infinie pour la messagerie. Les **changements organisationnels** touchent la répartition des tâches entre les deux développeurs.

9.2 Gestion des Versions

Le contrôle de versions est assuré par Git avec une stratégie de branches structurée. La branche `master` constitue la version de production stable. Les branches de fonctionnalités (`omar`, `redis`, et autres) isolent le développement. Les pull requests avec revue de code assurent la qualité avant fusion.

Les migrations Drizzle constituent un mécanisme de gestion du changement au niveau de la persistance. Chaque migration est un fichier SQL versionné (de 0000 à 0003) qui peut être appliquée et revertée. Cette approche garantit la cohérence du schéma entre les environnements.

9.3 Adaptation et Communication

L'architecture modulaire du projet a facilité l'adaptation aux changements. L'ajout du champ `logoUrl` dans la table `Launch` (migration 0002) n'a nécessité que la modification d'un seul fichier de schéma et de quelques lignes dans le composant frontend.

La communication autour des changements repose sur plusieurs canaux : la documentation technique (`README.md`, `AGENTS.md`) maintenue à jour, les messages de commit Git suivant une convention descriptive (`feat:`, `fix:`, `chore:`), et les discussions techniques documentées dans les pull requests.

Ce rapport a présenté l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de SaaS Directory. Du point de vue du cycle de vie, le modèle incrémental itératif adopté a permis de livrer des versions fonctionnelles successives, chacune apportant une valeur métier concrète.

La démarche orientée objet, matérialisée par le schéma Drizzle ORM de 20 tables et les composants React modulaires, assure la maintenabilité et l'évolutivité du système. Les cinq diagrammes UML (cas d'utilisation, classes, séquence, déploiement, GANTT) fournissent une documentation visuelle complète du projet.

La gestion de projet, illustrée par le WBS, le diagramme de GANTT, et l'estimation COCOMO, démontre la faisabilité d'une planification rigoureuse même dans un contexte académique. L'analyse des risques a permis d'anticiper et de mitiger les principales menaces techniques et sécuritaires.

L'utilisation d'une stack technologique moderne et cohérente — Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, Better Auth 1.6.9, PostgreSQL, et Redis — a grandement facilité le développement et garanti la qualité du livrable final.

Les perspectives d'évolution de SaaS Directory sont nombreuses. L'intégration complète de Stripe pour le paiement (le schéma contient déjà les champs `stripeCustomerId` et `stripeSubscriptionId`), l'ajout d'un système de recherche avancée, et le déploiement sur une infrastructure cloud avec Docker et Kubernetes constituent les axes de développement futurs.

- Barry Boehm, *Software Engineering Economics*, Prentice-Hall, 1977.
- Grady Booch, *Object-Oriented Analysis and Design with Applications*, 2nd Edition, Benjamin/Cummings, 1994.
- Jean-Raymond Abrial, *The B-Book : Assigning Programs to Meanings*, Cambridge University Press, 1996.
- ISO/IEC 12207 :2017, Systems and software engineering — Software life cycle processes.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 1028-2008, IEEE Standard for Software Reviews and Audits.
- IEEE Std 12207.0-1996, IEEE Standard for Information Technology — Software life cycle processes.
- Cours de Génie Logiciel et Conduite de Projet, Année Universitaire 2025-2026.
- Next.js Documentation, <https://nextjs.org/docs>, consulté en 2026.
- React Documentation, <https://react.dev>, consulté en 2026.
- Drizzle ORM Documentation, <https://orm.drizzle.team>, consulté en 2026.
- Better Auth Documentation, <https://www.better-auth.com>, consulté en 2026.
- Tailwind CSS Documentation, <https://tailwindcss.com>, consulté en 2026.
- PostgreSQL Documentation, <https://www.postgresql.org/docs>, consulté en 2026.
- Redis Documentation, <https://redis.io/documentation>, consulté en 2026.
- Vitest Documentation, <https://vitest.dev>, consulté en 2026.
- Playwright Documentation, <https://playwright.dev>, consulté en 2026.

Annexe A : Schema de la Base de Donnees Drizzle ORM

Cette annexe presente un extrait du schema Drizzle ORM (`src/lib/db/schema.ts`) illustrant la structure des tables principales.

```
// Tables Better Auth (authentification)
export const user = pgTable("user", {
  id: text("id").primaryKey(),
  name: text("name"),
  email: text("email").notNull().unique(),
  emailVerified: boolean("email_verified").default(false),
  image: text("image"),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const session = pgTable("session", {
  id: text("id").primaryKey(),
  userId: text("user_id").notNull().references(() => user.id),
  token: text("token").notNull().unique(),
  expiresAt: timestamp("expires_at").notNull(),
});

export const account = pgTable("account", {
  id: text("id").primaryKey(),
  userId: text("user_id").notNull().references(() => user.id),
  accountId: text("account_id").notNull(),
  providerId: text("provider_id").notNull(),
});

export const verification = pgTable("verification", {
  id: text("id").primaryKey(),
  identifier: text("identifier").notNull(),
  value: text("value").notNull(),
  expiresAt: timestamp("expires_at").notNull(),
});

// Table profile (1:1 avec user)
export const profile = pgTable("profile", {
```

```
id: serial("id").primaryKey(),
userId: text("user_id").notNull().references(() =>
  user.id).unique(),
displayName: text("display_name"),
bio: text("bio"),
website: text("website"),
twitter: text("twitter"),
github: text("github"),
avatarUrl: text("avatar_url"),
});

// Table launch (produit SaaS)
export const launch = pgTable("launch", {
  id: serial("id").primaryKey(),
  makerId: text("maker_id").notNull().references(() => user.id),
  title: text("title").notNull(),
  description: text("description"),
  logoUrl: text("logo_url"),
  upvoteCount: integer("upvote_count").default(0),
  createdAt: timestamp("created_at").defaultNow(),
});

// Table category
export const category = pgTable("category", {
  id: serial("id").primaryKey(),
  name: text("name").notNull(),
  slug: text("slug").notNull().unique(),
});

// Table plan (Free / Pro)
export const plan = pgTable("plan", {
  id: serial("id").primaryKey(),
  name: text("name").notNull(),
  priceMonthly: integer("price_monthly"),
  priceYearly: integer("price_yearly"),
  stripeProductId: text("stripe_product_id"),
  stripeMonthlyPriceId: text("stripe_monthly_price_id"),
  stripeYearlyPriceId: text("stripe_yearly_price_id"),
});
```

```
// Table subscription
export const subscription = pgTable("subscription", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull().references(() =>
    user.id).unique(),
  planId: integer("plan_id").notNull().references(() =>
    plan.id),
  status: text("status").notNull().default("active"),
  stripeCustomerId: text("stripe_customer_id"),
  stripeSubscriptionId: text("stripe_subscription_id"),
});
```

Annexe C : Dependances Principales

TABLE 2 – Dependances principales du projet SaaS Directory

Dependance	Version	Description
next	16.2.4	Framework React avec App Router
react	19.2.4	Bibliotheque UI
react-dom	19.2.4	Rendu DOM React
drizzle-orm	0.45.2	ORM type-safe pour PostgreSQL
better-auth	1.6.9	Authentification OAuth + 2FA
pg	8.20.0	Driver PostgreSQL
@tanstack/react-query	5.100.9	Gestion de cache et requetes
zod	4.4.3	Validation de schemas
framer-motion	12.38.0	Animations React
ioredis	5.10.1	Client Redis
resend	6.12.2	Service d'envoi d'emails
sonner	2.0.7	Notifications toast UI
tailwindcss	4.0.0	Framework CSS utilitaire
typescript	5.7.0	Typage statique
vitest	4.1.5	Tests unitaires et d'integration
@playwright/test	1.59.1	Tests end-to-end
eslint	9.0.0	Linting du code
eslint-config-next	16.2.4	Regles ESLint Next.js

Annexe D : Diagramme de Deploiement

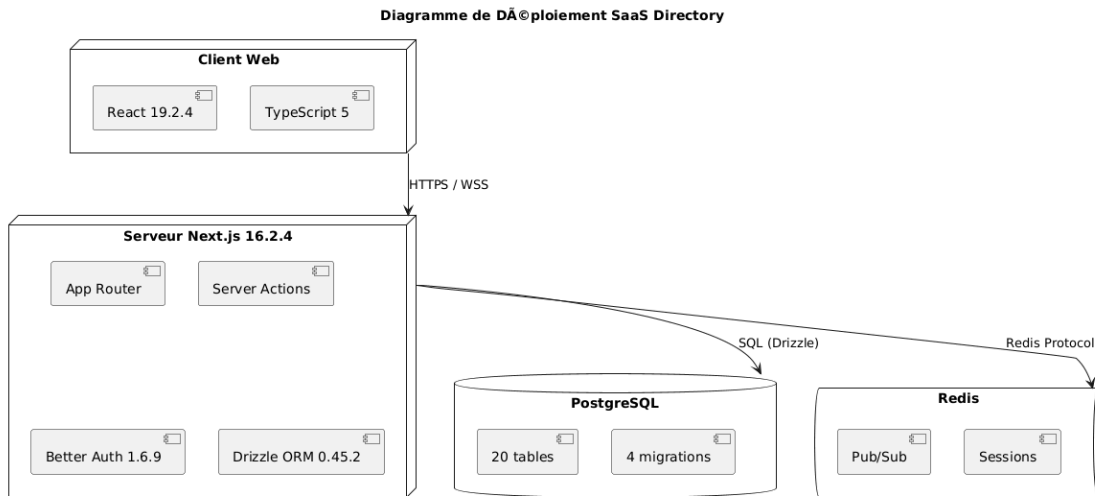


FIGURE 5 – Diagramme de Deploiement — Architecture Physique de SaaS Directory

La figure 5 decrit l'architecture physique du systeme. SaaS Directory repose sur une architecture distribuee sur trois environnements principaux : le client web (navigateur), la plateforme serveur (Next.js 16.2.4), et les services de donnees (PostgreSQL + Redis).

Le client web (React 19.2.4 / TypeScript 5) s'execute dans le navigateur et communique avec les fonctions serverless via HTTPS. Les Server Actions Next.js 16.2.4 interagissent avec la base de donnees PostgreSQL via Drizzle ORM 0.45.2. Redis est utilise comme cache cote serveur pour reduire la charge sur la base de donnees. L'authentification est assuree par Better Auth 1.6.9.